

Cherry — DealFlow Development Strategy

Architecture, engineering lifecycle, security, testing, releases and continuous improvement

Cherry — Working Company Document | September 2026

1. Development Philosophy

Build DealFlow as a continuously maintained product, not a one-time software project.

The engineering goal is not maximum initial complexity. It is a strong foundation that permits safe, fast, incremental evolution.

Core principle

Simple product experience + strong foundations + incremental architecture.

2. Product Development Lifecycle

Discover → Define → Design → Build → Verify → Release → Observe → Learn → Improve → Repeat.

Every significant feature should connect a customer problem to measurable expected value.

3. Architecture Principles

Modular domain architecture.

Clear separation of business logic, data access, interfaces and infrastructure.

Multi-tenant isolation from the foundation.

Strong identity and authorization boundaries.

Versioned APIs and controlled schema evolution.

Backward-compatible changes whenever practical.

Explicit migrations and rollback planning.

Auditability for sensitive business changes.

Configuration over customer-specific code.

Extensibility for future automation and AI.

4. Core Domain Model

Organization/Tenant.

User/Role/Permission.

Contact/Customer.

Lead.

Requirement.

Property/Project/Unit as the domain grows.

Activity/Interaction.

Follow-up/Task.

Site Visit.

Deal/Opportunity.

Document.

Payment/Commission as later domains.

Workflow/Automation.

Import Batch/Data Source.

Audit Event.

5. Data Architecture

Imported data must retain provenance and import context.

Active business records must have lifecycle state.

Changes to important entities should be traceable.

Migrations must be tested and reversible where technically feasible.

Deletion should be deliberate and governed; archival should be preferred where business history matters.

Data quality checks should be reusable rather than implemented separately for every import.

6. Engineering Workflow

Use issue tracking for epics, stories, bugs, technical tasks and releases.

Break work into small vertical slices that produce working functionality.

Keep branches focused and use consistent pull-request/review practices as the team grows.

Maintain architecture decision records for important technical decisions.

Keep documentation close to the system and update it as behavior changes.

7. Testing Strategy

Unit tests for business rules.

Integration tests for persistence and service boundaries.

API tests for critical workflows.

End-to-end tests for the primary customer journey.

Data migration/import tests.

Permission and tenant-isolation tests.

Regression tests for important customer workflows.

Performance testing for critical paths as usage grows.

8. Security Engineering

Authentication and authorization are foundational.

Every tenant's data must be isolated.

Use least privilege and explicit access rules.

Protect documents and sensitive customer information.

Maintain audit trails for sensitive actions.

Manage secrets and dependencies securely.

Continuously review vulnerabilities and security posture.

AI and automation must operate inside the same identity and permission boundaries as human users.

9. Observability and Reliability

Logs, metrics and traces should be designed into the system rather than added only after incidents.

Monitor errors, latency, job failures, imports, integrations and critical business workflows.

Use health checks and operational dashboards.

Establish backup, restore and recovery procedures before the product becomes business-critical.

Track incidents and convert recurring failures into engineering improvements.

10. Release and Version Management

Use predictable release practices for patches, minor improvements and major changes.

Use feature flags where gradual rollout reduces risk.

Use controlled database migrations.

Preserve compatibility for existing customer data and supported integrations.

Provide a deprecation process rather than silently removing behavior.

Maintain rollback or recovery procedures for production changes.

11. Continuous Improvement System

Collect customer feedback, support issues, analytics, incidents and product usage signals.

Convert repeated problems into product insights and backlog items.

Prioritize using customer value, frequency, strategic fit, revenue impact, risk and engineering cost.

After release, measure whether the change actually produced the expected outcome.

Retire or simplify features that create complexity without sufficient value.

12. AI Engineering Lifecycle

Start with assistive capabilities rather than autonomous actions.

Ground AI in trusted business data and explicit context.

Apply permission checks before exposing information or taking action.

Classify high-risk actions and require approval where appropriate.

Log AI-assisted actions and important decisions.

Evaluate AI quality continuously using representative business cases and regression tests.

13. Definition of Done

Requirement is clear.

UX is understandable.

Business rules are documented.

Security and permissions are addressed.

Tests are implemented.

Observability is present where appropriate.

Migration/data impact is understood.

Documentation is updated.

Release/rollback considerations are addressed.

Acceptance criteria pass in a production-like environment.

14. Completion Strategy for V1

Do not develop all modules in parallel.

Build one complete vertical business loop first: lead → requirement → property → match → follow-up → visit/next action → deal.

Then add surrounding capabilities.

A feature is considered production-ready only when the complete workflow works reliably, not when a screen exists.

Keep a strict V1 boundary so continuous improvement does not turn the initial launch into an endless project.

15. Long-Term Development Model

DealFlow should have an evergreen backlog and a recurring release cadence once customers are active.

Major capability areas can evolve independently while sharing common foundations.

Reusable engineering practices developed for DealFlow should become Cherry company standards where appropriate.

Future Cherry products should reuse the company's product-development discipline without forcing unrelated products into the DealFlow domain model.

16. Engineering North Star

Build software that is simple for customers, safe for their data, observable for operators, maintainable for engineers and extensible for future product capabilities.